WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

# Dossier d'innocuité
# WAC (Windows Artefacts Collector)

## Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

# WAC — Windows Artefacts Collector

Dossier d’innocuité

| | |
|---|---|
| Version du logiciel | 4.0.0 |
| Édition du document | octobre 2026 |
| Éditeur | Centre d’Excellence Cyberdéfense Aérospatiale (CEC) |
| | École de l’air et de l’espace, base aérienne 701, Salon-de-Provence |

Ce document établit ce que WAC fait à la machine qu’il examine : chaque opération, la trace qu’elle laisse, et ce qu’il ne fait jamais. Il donne les preuves de ces affirmations — la table des fonctions que `WAC.exe` peut appeler, les contrôles qui refusent une compilation fautive, la mesure faite sur une machine de test — et la procédure qui permet à chacun de les refaire.

# Table des matières


**1 Objet** **2**

1.1 Ce que « innocuité » veut dire ici . . . . . 2

1.2 Ce que ce document ne couvre pas . . . . . 2

**2 Inventaire des traces** **3**

2.1 L'intervention . . . . . 3

2.2 L'exécution de WAC.exe par Windows . . . . . 3

2.3 Mesuré face à KAPE et aux outils EZ . . . . . 4

2.4 Les opérations de WAC . . . . . 5

2.5 Ce que WAC ne fait jamais . . . . . 8

**3 Les preuves** **9**

3.1 La table des imports de WAC.exe . . . . . 9

3.2 Les garde-fous de la compilation . . . . . 10

3.3 Le journal d'investigation . . . . . 11

**4 Procédure de validation** **12**

4.1 Étape 1 : le binaire livré est celui des sources . . . . . 12

4.2 Étape 2 : la table des imports . . . . . 12

4.3 Étape 3 : la mesure sur une machine de test . . . . . 12

4.4 Étape 4 : le journal d'une collecte réelle . . . . . 13

**5 Limites** **14**

CHAPITRE 1

# Objet

## 1.1 Ce que « innocuité » veut dire ici

Une collecte se fait sur une machine qui est aussi un élément de preuve. Un outil qui la modifie détruit une partie de ce qu'il vient chercher, et mêle ses propres traces à celles de l'activité étudiée. WAC est dit *inoffensif* au sens suivant :

1. il **n'écrit rien** sur les volumes de la machine examinée — ni fichier, ni registre, ni événement dans un journal ; ce qu'une surveillance en place (Sysmon, audit, EDR) enregistre de son exécution est décrit au chapitre 2 ;
2. il **ne modifie aucune métadonnée** des fichiers qu'il lit : ni date d'accès, ni numéro de changement du journal NTFS ;
3. il **n'émet rien sur le réseau** et ne résout aucun nom ;
4. il **n'installe rien et ne persiste pas** : ni service, ni pilote, ni tâche, ni clé de démarrage ; à sa sortie il ne reste de lui que les traces que Windows produit pour tout programme lancé ;
5. il **n'altère pas le fonctionnement** de la machine : aucun processus ouvert, aucun service arrêté ou reconfiguré, aucun privilège modifié ;
6. ce qu'il ne peut pas éviter est **énuméré**, ici et, collecte par collecte, dans le journal `investigation.json`.

## 1.2 Ce que ce document ne couvre pas

— La justesse des données collectées : c'est l'objet du *Dossier de validation*.

— L'exploitation courante de l'outil : c'est l'objet du *Manuel d'utilisation*.

— La mise à jour des listes de confiance (`--update-trust`) : elle s'exécute sur le poste d'analyse, jamais sur la machine examinée, et c'est le seul mode qui ouvre une connexion réseau.

CHAPITRE 2

# Inventaire des traces

Les traces se rangent en trois familles : celles de l'intervention elle-même (brancher un support, ouvrir une session), celles que Windows produit pour tout programme lancé, et celles des opérations propres à WAC. Les deux premières existent quel que soit l'outil ; elles sont listées pour que l'analyste les reconnaisse au lieu de les prendre pour l'activité étudiée.

## 2.1 L'intervention

| Action | Trace |
|---|---|
| Brancher le support de collecte | Clés `USBSTOR`, `USB`, `MountedDevices`, `Windows Portable Devices`, `VolumeInfoCache` ; fichier `setupapi.dev.log` ; journaux `Microsoft-Windows-Partition/Diagnostic` (1006), `Kernel-PnP`, `DeviceSetupManager`. |
| Ouvrir une session | Événements Security 4624 et 4672 (privilèges spéciaux) ; pour un compte jamais connecté sur la machine, création de son profil. |
| Élever l'invite de commandes | Exécution de `consent.exe` (contrôle de compte d'utilisateur) et ses propres traces d'exécution. |
| Lancer `cmd.exe` | Traces d'exécution de `cmd.exe` (section suivante). |

## 2.2 L'exécution de WAC.exe par Windows

Windows les produit pour tout programme ; WAC n'y peut rien et ne tente pas de les effacer — les effacer serait écrire sur la machine.

| Artefact | Trace |
|---|---|
| Prefetch | `C:\Windows\Prefetch\WAC.EXE-<empreinte>.pf`, écrit par Windows quelques secondes après le lancement, puis à la fin — pour un `WAC.exe` lancé du disque de la machine : lancé depuis une clé USB, mesuré sous Sysmon, il n'en a pas eu. |

| Artefact | Trace |
|---|---|
| Amcache, ShimCache | Une entrée pour `WAC.exe` (chemin, taille, date, empreinte SHA-1 pour Amcache). Mesurée : l'entrée Amcache est écrite pendant la collecte, celle du ShimCache à l'arrêt de la machine, sous la forme `SIGN. MEDIA=<signature> WAC\WAC.exe` pour une clé. |
| BAM | Une valeur pour `WAC.exe` sous la clé de l'utilisateur, datée de la fin de l'exécution — là encore pour un lancement depuis le disque de la machine : aucune depuis la clé USB. |
| SRUM | Consommation de ressources du processus, agrégée par Windows. |
| Journaux | Security 4688 et 4689 si l'audit de création de processus est actif ; Sysmon 1 si Sysmon est installé ; télémétrie de l'antivirus ou de l'EDR (analyse de `WAC.exe`, et, pour un EDR, signalement possible de l'ouverture du volume en lecture brute). |
| Mémoire | Les pages de WAC occupent la mémoire pendant la collecte et peuvent en évincer d'autres vers le fichier d'échange. |
| RADAR | Si la mémoire de `WAC.exe` passe le seuil de Windows, son détecteur de fuites le note : une valeur sous `HKLM\SOFTWARE\Microsoft\RADAR\HeapLeakDetection\ DiagnosedApplications\WAC.exe` (`LastDetectionTime`) et l'événement 1014 de `Microsoft-Windows-Resource-Exhaustion-Resolver/ Operational`. Vu deux fois sur la machine de test, en septembre et octobre 2026, aucune lors des cycles de la version 3.0.0. |

> **À retenir**
>
> WAC collecte ces mêmes artefacts : il se retrouve donc dans ses propres résultats (Prefetch de la collecte suivante, BAM, Amcache). C'est attendu, et daté : l'intervalle `Collection.StartUtc` — `EndUtc` de `investigation.json` permet de les isoler.

## 2.3 Mesuré face à KAPE et aux outils EZ

Sur un clone de la machine de test, figé avec Sysmon, chaque outil est lancé depuis une clé USB à partir du même état ; le journal des modifications de C:, borné avant et après l'outil, et Sysmon disent ce qu'il a écrit, et qui l'a écrit ; après l'arrêt, les ruches du disque sont relues hors ligne et confrontées à celles de l'état figé, clé par clé et valeur par valeur (document de comparaison, chapitre « Traces laissées sur la machine examinée »). Mesure des 29 et 30

octobre 2026, sur la version 3.0.0 et la correction de la liste des modules décrite plus bas.

| Outil | Ce que ses processus écrivent sur la machine |
|---|---|
| WAC | Rien : aucun fichier, aucune valeur de registre. |
| KAPE (cibles) | 4 valeurs `ZoneMap` d'Internet Settings dans la ruche de l'utilisateur. |
| KAPE et ses modules | Les mêmes valeurs, réécrites par 14 processus ; 8 clés de traçage RAS (`HKLM\SOFTWARE\Microsoft\Tracing`) créées par 4 outils ; 28 fichiers sur C : (miettes du runtime .NET, fichiers sentinelles dans le Temp de l'utilisateur). |
| Outils EZ en direct | Aucune valeur ni aucun fichier, mais des ouvertures en création de clés de stratégie et de réseau. |

Les ruches relues hors ligne le confirment : ce qui y change pendant la collecte de WAC, hors ce que le témoin change aussi, est l'entrée Amcache de `WAC.exe` et l'activité de Windows dans la fenêtre de mesure (WaaSMedic, Defender, tâches planifiées, WMI, Windows Update, OneDrive), que Sysmon attribue à d'autres processus. Aucun outil n'a fait écrire d'événement dans les journaux de Windows eux-mêmes. Mais tous sont enregistrés par la surveillance installée, comme le prévoit la section précédente : Sysmon a noté, pour WAC, son processus, ses 3 lectures brutes (C:, la partition EFI, le disque physique), les 25 532 fichiers qu'il a créés, tous sur la clé, et les processus qu'il a ouverts — en lecture limitée par Terminal Services, puis en requête et lecture mémoire pour leur liste de modules ; l'audit de création de processus, inactif sur le clone, aurait inscrit un 4688 et un 4689. Pour tous, Windows ajoute une entrée Amcache pendant l'exécution et une entrée ShimCache à l'arrêt ; tous lisent les volumes en brut. KAPE et les outils EZ demandent en outre le runtime .NET 9 sur la machine examinée.

## 2.4 Les opérations de WAC

Chaque opération est consignée dans `investigation.json` avec, dans le champ `Footprint`, la trace qu'elle est censée laisser. Le tableau ci-dessous les reprend toutes.

| Opération | Trace |
|---|---|
| Lecture directe du volume (`\\.\C:`) | Un descripteur ouvert en **lecture seule**, sans cache (`FILE_FLAG_NO_BUFFERING`), sur chaque volume effectivement lu. Aucun fichier n'est ouvert par le système : aucune date d'accès modifiée, aucune entrée du journal USN, aucun verrou. Si l'audit d'accès aux objets est configuré sur le volume : Security 4656/4663. Sans cache, la lecture ne remplace pas non plus le contenu du cache de fichiers de la machine. |

| Opération | Trace |
|---|---|
| Ruches du registre | Copiées par la lecture directe, puis lues sur la copie. Aucune clé du registre vivant n'est ouverte : `WAC.exe` n'importe aucune fonction `Reg*`. Le rejeu des journaux de transaction se fait sur la copie, avec un journal d'annulation. Dans la ruche SECURITY, sont lus les noms et dates des secrets LSA (jamais leur valeur), les comptes de connexions en cache (jamais leur contenu) et les droits des comptes. Dans la ruche SAM, les condensats des mots de passe sont déchiffrés et les questions de réinitialisation lues en clair : une collecte voulue, pour que WAE repère les mots de passe faibles. Tout se fait sur la copie, en lecture seule ; la machine examinée n'est pas touchée, et un condensat est à sens unique. |
| Journaux d'événements (`--events`) | Lus par la lecture directe : le service de journalisation n'est pas sollicité, et cette lecture ne produit aucun événement. |
| Volumes FAT et exFAT | Lus en brut par le périphérique du volume (`\\.\X:`), en lecture seule, volumes fixes comme amovibles : secteur de démarrage, FAT et répertoires, jamais un fichier ouvert, aucune date d'accès modifiée. Le volume qui reçoit la collecte n'est pas lu. |
| Secteurs de démarrage (`--boot_records`) | Lus en brut par le périphérique de chaque disque physique (`\\.\PhysicalDriveN`), ouvert en lecture seule : premier mébioctet, premiers secteurs de chaque partition, fichiers de la partition système EFI par le lecteur FAT de WAC — la partition n'est pas montée, aucune lettre ne lui est donnée. Le disque du support de collecte n'est pas lu, sauf s'il porte le système. |
| Caches de jetons Entra ID (`--cloud_identity`) | Lus par la lecture directe et prélevés tels quels, chiffrés par DPAPI : WAC ne les déchiffre pas et ne s'en sert pas ; aucun service en nuage n'est contacté. |
| Annuaire et stratégies (`--active_directory`, `--group_policies`) | Lus par la lecture directe : `ntds.dit`, ses journaux et `SYSVOL`. Aucune requête LDAP, aucune réplication : `WAC.exe` n'importe pas de bibliothèque LDAP, et le contrôleur n'enregistre donc pas d'accès à l'annuaire au nom de WAC. Le rejeu des journaux se fait sur la copie de travail. |
| Serveurs web (`--web_server`) | Configuration, sites, journaux et bases lus par la lecture directe. Ni le serveur web ni aucun moteur de base n'est interrogé, arrêté ou redémarré ; une base vivante est prise telle que le disque la porte. |

| Opération | Trace |
|---|---|
| Binaires (`--binary`) | Lus par la lecture directe ; signatures et catalogues vérifiés en mémoire. Aucun magasin de certificats ouvert, aucune vérification de révocation en ligne. |
| Processus | Un instantané du noyau (`CreateToolhelp32Snapshot`, la liste des processus seule) et une requête au service Terminal Services (`WTSEnumerateProcessesW`) pour les propriétaires, qui ouvre chaque processus en lecture limitée (`0x1000`), LSASS compris. |
| Modules des processus | Chaque processus ouvert en requête et lecture mémoire seules (`OpenProcess`, `PROCESS_QUERY_INFORMATION` et `PROCESS_VM_READ`, accordés en `0x1410`), sa liste lue par psapi : ni écriture, ni fil, ni injection. Ceux que le noyau protège (LSASS sous protection LSA, services, antivirus) le refusent, et le refus est consigné. Jusqu'à la version 3.0.0, l'instantané de modules de Toolhelp ouvrait lui-même chaque processus en accès complet (`PROCESS_ALL_ACCESS`, mesuré sous Sysmon) : remplacé. |
| Sessions | Une requête en lecture à LSASS (`LsaEnumerateLogonSessions`, `LsaGetLogonSessionData`). |
| État des services | Une énumération en lecture seule du gestionnaire de services, droits `SC_MANAGER_ENUMERATE_SERVICE \| SC_MANAGER_CONNECT`. Aucun service n'est ouvert ; la configuration vient de la ruche copiée. |
| État réseau (`artefacts.network`) | Lecture en mémoire des tables de la pile TCP/IP par `iphlpapi.dll`, chargée à ce moment seulement. Aucun paquet émis, aucun nom résolu. |
| Horloge et démarrage | Requêtes en mémoire au noyau (`NtQuerySystemInformation`, `GetTickCount64`). |
| Réglages de la machine | Lecteur système, page de code, fuseau horaire, volumes et numéros de série : requêtes en mémoire, sans ouverture de fichier. |
| Jeton du processus | Lecture des privilèges de WAC lui-même, pour vérifier les droits d'administrateur. Aucun privilège n'est activé ni modifié. |
| Écriture des résultats | Sur le dossier `--output` uniquement, relatif au répertoire courant — le support de collecte. |

**Limite**

WAC écrit dans le dossier `--output` du répertoire courant. `--output` (et `output:` de `wac.yml`) n'est qu'un nom de dossier : sans séparateur, sans lecteur (`C:case`), autre que `..` ;

tout le reste est refusé avant toute collecte, pour qu'il ne puisse pas désigner le volume examiné. Lancé depuis un dossier du disque examiné, en revanche, il y écrirait ses résultats : fichiers créés, enregistrements du `$MFT` et du journal USN, blocs écrasés. **Lancez-le depuis le support de collecte**, qui doit être le répertoire courant.

## 2.5 Ce que WAC ne fait jamais

— Créer un cliché instantané (VSS), appeler COM ou WMI : aucune de ces bibliothèques n'est importée. Les clichés existants sont lus en brut sur le volume (catalogue et magasins de la zone de différences), par le même accès en lecture seule que le reste : aucun n'est monté, exposé ni demandé au service.

— Récupérer un fichier supprimé par un outil, ou par une écriture sur le volume : le contenu des fichiers supprimés est lu en brut (enregistrements de la `$MFT`, `$Bitmap`, clusters libres, blocs d'index des répertoires, et au niveau 3 tout l'espace libre ; volumes FAT et exFAT par leur périphérique), par le même accès en lecture seule que le reste, et copié sur le support de collecte seulement. Un cluster en usage n'est pas lu.

— Ouvrir une clé du registre vivant, ou le vider sur le disque (`RegFlushKey`).

— Charger un pilote, créer ou modifier un service, une tâche planifiée, une clé de démarrage.

— Ouvrir un processus, lire sa mémoire, injecter du code.

— Lancer un processus : aucune fonction qui en crée un n'est importée (`system`, `CreateProcess`, `ShellExecute`, `WinExec`). L'effacement de l'écran au démarrage lançait `cmd.exe /c cls`, avec ses traces d'exécution ; la mesure comparée des traces laissées l'a montré, et l'écran est désormais effacé par l'API de la console.

— Activer un privilège (`AdjustTokenPrivileges`) : la lecture directe d'un volume ne demande que les droits d'administrateur.

— Ouvrir une connexion réseau, résoudre un nom de compte auprès du contrôleur de domaine (`LookupAccountSid`) : les comptes sont nommés hors ligne, depuis les ruches copiées.

— Écrire dans un fichier temporaire du système.

CHAPITRE 3

# Les preuves

## 3.1 La table des imports de WAC.exe

Un programme Windows ne peut appeler une fonction du système que s'il l'importe, ou s'il charge sa bibliothèque dynamiquement. La table des imports de `WAC.exe` est donc la liste exhaustive de ce qu'il peut demander au système — hors les deux chargements dynamiques décrits plus bas. Elle se lit sans exécuter le programme, sous Linux :

```
$ x86_64-w64-mingw32-objdump -p WAC.exe | grep -E 'DLL Name|^\s+[0-9a-f]+\s+[0-9]+ '
```

Hors `msvcrt.dll` (la bibliothèque C) :

**ADVAPI32.dll**
: `OpenSCManagerW`, `EnumServicesStatusExW`, `CloseServiceHandle` (état des services); `OpenProcessToken`, `GetTokenInformation`, `GetUserNameW`, `GetLengthSid`, `IsValidSid` (le jeton de WAC lui-même); `AllocateAndInitializeSid`, `CheckTokenMembership`, `FreeSid` (l'appartenance au groupe des administrateurs, seule élévation que connaît Windows XP). Aucune fonction `Reg*`, aucune fonction de modification de service.

**Secur32.dll**
: `LsaEnumerateLogonSessions`, `LsaGetLogonSessionData`, `LsaFreeReturnBuffer`.

**WTSAPI32.dll**
: `WTSEnumerateProcessesW`, `WTSFreeMemory`.

**PSAPI.DLL**
: `EnumProcessModules`, `GetModuleFileNameExW` : la liste des modules de chaque processus, ouvert par `OpenProcess` en requête et lecture mémoire.

**bcrypt.dll**
: `BCryptOpenAlgorithmProvider`, `BCryptSetProperty`, `BCryptGetProperty`, `BCryptGenerateSymmetricKey`, `BCryptDecrypt`, `BCryptDestroyKey`, `BCryptCloseAlgorithmProvider` : le déchiffrement AES des condensats de la SAM, à la conversion seule. Du calcul en mémoire, sans fichier ni registre.

**KERNEL32.dll**
: Fichiers, mémoire, fils d'exécution, heure, console, volumes, instantané et ouverture des processus. Les fonctions qui écrivent ou suppriment (`DeleteFileW`, `MoveFileExW`, `RemoveDirectoryW`, `CreateHardLinkW`, `SetEndOfFile`) servent au dossier de sortie, ou appartiennent à la bibliothèque C++ liée statiquement (`std::filesystem`) sans que WAC les appelle ailleurs : chaque suppression ou renommage du code source porte sur le dossier de sortie, sa copie de travail ou son dossier d'attente.

**Chargements dynamiques.** Trois bibliothèques seulement, depuis `System32` (`LOAD_LIBRARY_SEARCH_SYSTEM32` ou, sous un Windows qui ne connaît pas ce drapeau, leur chemin absolu dans ce dossier) : `iphlpapi.dll`, pour l'état réseau, quand `artefacts.network` est demandé ; `winhttp.dll`, pour `--update-trust` seul, et `esent.dll`, qui rejoue les journaux d'une base ESE copiée (WebCache, SRUM, BITS) dans la copie de travail, à la conversion seule — ni l'une ni l'autre ne s'exécute sur la machine examinée : WAC y collecte, et n'y convertit jamais.

**`WAC32.exe`**, le WAC 32 bits (Windows XP SP1 à Windows 8 x86), est construit des mêmes sources et importe les mêmes fonctions, sauf celles que XP n'a pas : les quatorze fonctions venues avec Vista (dont toute `bcrypt.dll`) y sont définies par WAC lui-même, qui appelle la fonction de Windows quand elle existe et la remplace sinon par ce que XP offre. `bcrypt.dll` est alors chargée depuis `System32`, à la conversion seule. S'y ajoutent `CreateSemaphoreW`, `ReleaseSemaphore` (les fils de la bibliothèque C++ 32 bits) et `LoadLibraryA`, que son code de démarrage n'appelle que pour une bibliothèque d'exécution déjà présente dans le processus. La compilation confronte chaque import de `WAC32.exe` aux exportations de Windows XP SP1.

## 3.2 Les garde-fous de la compilation

`build-windows.sh` refuse de produire un `WAC.exe` qui enfreindrait ces règles. Après l'édition de liens, il inspecte la table des imports et échoue :

— si `WAC.exe` importe une fonction de résolution de noms qui interroge le domaine (`LookupAccountSid`, `LookupAccountName`, `LsaLookupSids`, `LsaLookupNames`) ;

— s'il importe statiquement une bibliothèque réseau (`winhttp`, `wininet`, `ws2_32`, `wsock32`, `iphlpapi`) ;

— s'il dépend d'une bibliothèque d'exécution MinGW : l'exécutable doit se suffire, pour ne rien déposer à côté de lui.

Avant même de compiler, il refuse un compilateur dont le *modèle de threads* n'est pas « win32 ». Un gcc en threads « posix » — celui de MSYS2 sous Windows — intègre winpthreads à `WAC.exe`, et avec lui 38 imports que WAC n'appelle jamais : `SetSystemTime`, `SetThreadContext`, `SuspendThread`, `TerminateProcess`, `SetProcessAffinityMask`… Ils ne changent rien à ce que fait WAC, mais ils élargissent ce qu'il *pourrait* demander au système, et ce sont précisément les noms qu'un outil de défense ou un analyste remarque sur la machine examinée. Mesuré : avec le gcc de Debian (Linux) ou celui de MinGW-Builds (Windows), tous deux en threads « win32 » et sur msvcrt, ces imports sont absents, et les deux tables ne diffèrent que de huit fonctions d'entrée-sortie de la bibliothèque C. « win32 » nomme le modèle de threads, pas une architecture : `WAC.exe` est x86_64 dans tous les cas.

La compilation est reproductible : les mêmes sources donnent le même `WAC.exe`, octet pour octet, et `SHA256SUMS` l'atteste.

## 3.3 Le journal d’investigation

Chaque collecte écrit, dans `investigation.json`, la liste ordonnée de ses opérations, chacune avec sa cible, son résultat et son champ `Footprint`. Un écart entre ce que dit ce dossier et ce que dit le journal d’une collecte réelle se voit donc collecte par collecte, et non seulement lors d’une campagne de test.

CHAPITRE 4

# Procédure de validation

Cette procédure est à refaire à chaque version, et à chaque modification qui touche un appel au système. Les deux premières étapes se font sur le poste d'analyse, sans machine Windows.

## 4.1 Étape 1 : le binaire livré est celui des sources

```
$ ./build-windows.sh
$ sha256sum -c bin/SHA256SUMS
```

Attendu : la compilation se termine sans erreur, sans avertissement, et affiche les trois contrôles : « Static C++ runtime », « No network name resolution imported », « No network library imported ». `WAC.exe` compilé et celui de `bin` ont la même empreinte.

## 4.2 Étape 2 : la table des imports

```
$ x86_64-w64-mingw32-objdump -p bin/WAC.exe | grep -E 'DLL Name|^\s+[0-9a-f]+\s+[0-9]+ '
```

Attendu : les cinq bibliothèques et les fonctions de la section 3, et aucune autre. Une fonction nouvelle s'explique dans ce dossier avant la livraison.

## 4.3 Étape 3 : la mesure sur une machine de test

Le cycle automatique (`vmtest/run-wac-test.sh`, voir le *Guide des tests*) lance WAC dans une machine virtuelle (Windows 11, et Vista à 8.1) et vérifie, sur des fichiers témoins que WAC lit, qu'aucune de leurs métadonnées ne change :

1. avant la collecte, le cycle crée des fichiers témoins que WAC lit forcément (un document à macro et un script, que `--binary` prélève) et relève, par PowerShell, leurs quatre dates et le numéro de leur dernier changement dans le journal NTFS (`fsutil usn readdata`) ;
2. il lance la collecte ;
3. il relève à nouveau les mêmes valeurs : elles doivent être identiques ;
4. pour prouver que le contrôle sait échouer, il lit ensuite un témoin par l'API (`Get-Content`) et vérifie que son numéro USN, lui, change quand Windows met à jour sa date d'accès.

Le résultat figure dans le compte rendu du cycle (`harmlessness.txt`). Le 29 octobre 2026, sur le code de la version 3.0.0 : témoins inchangés sous Windows 11 et sous Windows 8.1. Sous Windows 11, la lecture par Windows a changé la date d'accès : le contrôle sait échouer. Windows 8.1 ne tenait pas les dates d'accès du volume : là, le contrôle ne prouve que le numéro USN. Le 4 novembre 2026, sur le code de la 4.0, sur un contrôleur de domaine sous Server 2022, les dates d'accès activées pour la mesure (`fsutil behavior set disablelastaccess 0`, rétabli ensuite) : témoins inchangés, alors qu'une lecture par Windows a changé la date d'accès. Le contrôle sait donc y échouer aussi. Sur ce contrôleur tel que Windows le laisse, les dates d'accès sont éteintes et le contrôle ne prouve que le numéro USN. Les 7 et 8 novembre 2026, sur le code de la 4.0.0, WAC lancé depuis une clé USB comme sur le terrain : témoins inchangés à chaque cycle complet, sous Windows 11 et sous chaque serveur de Server 2008 R2 à 2025, Core et rôles compris ; sous Windows 11, une lecture par Windows a encore changé la date d'accès. Les 12 et 13 novembre 2026, sur le code de la 5.0.0 (traces d'applications comprises, lues en brut comme le reste) : témoins inchangés à chaque cycle complet, sous Windows 11, Server 2025 (contrôleur de domaine compris), les rôles de 2022 et Server 2008 R2 ; sous Windows 11, une lecture par Windows a encore changé la date d'accès, sur les serveurs les dates d'accès sont éteintes et le contrôle ne prouve que le numéro USN. La mesure sous Sysmon, face à KAPE, reste celle du poste sous Windows 11 : elle n'a pas été refaite sur un serveur.

## 4.4 Étape 4 : le journal d'une collecte réelle

Sur une collecte réelle, relire `investigation.json` : chaque opération doit appartenir au tableau du chapitre 2, avec le `Footprint` correspondant. Une opération inconnue de ce dossier est un défaut à signaler.

CHAPITRE 5

# Limites

**Dossier de sortie**
`--output` ne peut pas quitter le répertoire courant, mais WAC ne vérifie pas que celui-ci est sur un autre volume que ceux qu'il examine : c'est à l'opérateur de le lancer depuis le support de collecte.

**Surveillance**
Un EDR ou une politique d'audit peut journaliser l'ouverture du volume en lecture brute, l'énumération des processus et des sessions, et l'ouverture de chaque processus en lecture mémoire pour sa liste de modules (Sysmon, événement 10 ; LSASS, quand il n'est pas protégé, en fait partie). Ces journaux sont écrits par la machine, non par WAC ; ils sont attendus.

**Mémoire**
WAC ne capture pas la mémoire, mais il en occupe : une page évincée vers le fichier d'échange pendant la collecte n'est pas une trace sur le disque que WAC lit, mais elle change l'état de la mémoire que capturerait un outil lancé ensuite. Capturer la mémoire, si elle est nécessaire, **avant** WAC.

**Mesure**
L'étape 3 porte sur des fichiers témoins d'une machine virtuelle : elle établit le comportement de la lecture, pas l'absence de toute écriture de Windows lui-même pendant la collecte.